



ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**Hospital Management Database Application**” is submitted by

A ABHIRAM	(26B21CS010)
G BENJAMIN	(26B21CS030)
CH ENOSH	(26B21CS021)
B ANIL KUMAR	(26B21CS099)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

Project Guide

Mrs. K Vydehi, M.Tech., (Ph. D),
Assistant Professor,
Department of CSE

Head of the Department

Dr. Phani Sridhar Addepalli, Ph. D,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled “**Hospital Management Database Application**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

A ABHIRAM	(26B21CS010)
G BENJAMIN	(26B21CS030)
CH ENOSH	(26B21CS021)
B ANIL KUMAR	(26B21CS099)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. K Vydehi, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor, Dr Hari Jyothula, Associate Professor and Dr Pennada Siva Satya Prasad, Assistant Profeesor**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

TABLE OF CONTENTS

S. No.	Chapter / Section	Page No.
1	INTRODUCTION	1
1.1	Brief Information about the Project	1
1.2	Motivation and Contribution of the Project	1
1.3	Objectives of the Project	1
1.4	Scope of the Project	1
2	SYSTEM ANALYSIS	2 - 3
2.1	Existing System	2
2.2	Proposed System	2
2.3	Functional Requirements	2
2.4	Non-Functional Requirements	3
2.5	Hardware and Software Requirements	3
3	DATABASE DESIGN	4
3.1	Major Entities	4
3.2	Relationships	4
3.3	ER Model	4
4	ER DIAGRAM	5
5	RELATIONAL SCHEMA	6
6	TABLE DESIGN	7 - 9
6.1	Patient Table	7
6.2	Department Table	7
6.3	Doctor Table	7
6.4	Doctor_Phone Table	7
6.5	Employees Table	8
6.6	Receptionist Table	8
6.7	Nurse Table	8
6.8	Appointment Table	8
6.9	Admission Table	9
6.10	Billing Table	9
6.11	Keys and Constraints	9
7	NORMALIZATION	10 - 12

7.1	First Normal Form (1NF)	10
7.2	Second Normal Form (2NF)	10 - 11
7.3	Third Normal Form (3NF)	11 - 12
8	SQL IMPLEMENTATION	13 - 15
8.1	DDL – Data Definition Language	13 - 14
8.2	DML – Data Manipulation Language	15
8.3	DQL – Data Query Language	15
9	SQL QUERIES AND OUTPUTS	16 - 23
10	CONCLUSION	24
11	FUTURE ENHANCEMENTS	25
12	REFERENCES	25

ABSTRACT

The Hospital Management System (HMS) is a DBMS project designed to store, organize, and manage hospital information efficiently. It manages data related to patients, doctors, nurses, receptionists, departments, appointments, and treatments in a structured manner.

The system helps reduce manual record-keeping, data duplication, and errors, while making information retrieval faster and easier. The database is designed using an Entity-Relationship (ER) model and implemented through relational tables using primary keys, foreign keys, and integrity constraints.

SQL is used to create tables, insert records, update existing data, and retrieve information from the database. The project demonstrates important DBMS concepts such as normalization, entity relationships, relational integrity, and SQL operations.

Overall, the Hospital Management System provides a structured and efficient approach to managing hospital records while demonstrating the practical application of Database Management System concepts in the healthcare domain.

1. INTRODUCTION

1.1 Brief Information about the Project

The Hospital Management System (HMS) is developed to manage the day-to-day information of a hospital in an organized manner. It handles patient registration, doctor details, appointments, departments, admissions, billing, nurses, and receptionists. The system keeps all hospital records in one place, making it easier to add, update, search, and manage information. It helps the hospital maintain accurate records and makes common hospital activities more organized and efficient.

1.2 Motivation and Contribution of Project

The project is motivated by common problems in hospital management such as maintaining patient records manually, difficulty in tracking appointments and admissions, duplicate information, and time-consuming record searches. A centralized database provides a systematic way to manage hospital-related information and maintain proper relationships between different hospital entities.

- Centralizing hospital records in one database.
- Reducing duplicate and inconsistent information.
- Maintaining relationships between patients, doctors, departments, and other records.
- Supporting faster searching and retrieval of hospital information.
- Providing a foundation for patient, appointment, admission, billing, doctor, nurse, and receptionist

1.3 Objectives of the project

- To manage patient records efficiently.
- To maintain doctor, nurse, and receptionist details.
- To manage appointments, admissions, and billing.
- To maintain proper relationships between hospital records.
- To reduce duplicate and inconsistent data.
- To provide easy searching and retrieval of hospital information.

1.4 Scope of the Project

The scope covers database-oriented management of hospital information. The core scope includes patients, doctors, departments, appointments, admissions, billing, nurses, and receptionists. The database can be used as the backend for a hospital management application and can be extended later with pharmacy management, laboratory management, medical records, online appointments, and billing/payment systems.

2.SYSTEM ANALYSIS

2.1 Existing System

- Manual record maintenance.
- Multiple Excel sheets or disconnected files.
- Duplicate records and inconsistent information.
- Time-consuming searches.
- Difficulty in generating consolidated reports.
- Higher possibility of data-entry errors.

2.2 Proposed System

The proposed system uses a centralized relational database to organize hospital information into related tables. It uses primary keys, foreign keys, uniqueness rules, NOT NULL conditions, and CHECK constraints where appropriate. The system supports patient, doctor, department, appointment, admission, billing, nurse, and receptionist management.

- Centralized database
- Patient management
- Doctor management
- Department management
- Appointment management
- Admission management
- Billing management
- Nurse and receptionist management
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements

Module	Functional Requirement
Patient Management	Manage patient details and contact information.
Appointment Management	Schedule and manage patient appointments with doctors.
Doctor Management	Maintain doctor details, specialization, and consultation fees.
Department Management	Manage department details and locations.
Admission Management	Manage patient admission, discharge, and bed details.
Billing Management	Maintain patient bills, amounts, dates, and payment status.
Employee Management	Maintain employee details and shift timings.
Receptionist Management	Manage receptionist details and desk numbers.
Nurse Management	Maintain nurse specialization and experience details.

2.4 Non-Functional Requirements

- Performance – Hospital information should be retrieved and processed efficiently.
- Integrity – Invalid references and duplicate records should be prevented.
- Security – Access to patient and hospital records should be properly controlled.
- Usability – Hospital information should be easy to manage and understand.
- Reliability – Hospital records should remain accurate and consistent.
- Scalability – The system support the growth of patients, doctors, appointments, and other hospital records.

2.5 Requirements specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used by the implementation.

3.DATABASE DESIGN

3.1 Major Entities

Entity	Description
Patient	Stores patient identity, name, DOB, gender, and phone number.
Department	Stores department name and hospital location.
Doctor	Stores doctor name, specialization, department, and consultation fee.
Doctor Phone	Stores the multivalued phone numbers of doctors.
Employees	Stores common employee attributes and acts as the supertype.
Receptionist	Stores receptionist-specific desk information.
Nurse	Stores nurse-specific specialization and experience.
Appointment	Stores appointment information linking a patient and doctor.
Admission	Stores patient admission, discharge, and bed information.
Billing	Stores patient billing information and payment status.

3.2 Relationship Summary

Relationship	Cardinality / Description
Patient → Appointment	One patient can schedule one appointment
Appointment → Doctor	One Doctor handles one patient
Doctor → Department	Many doctors can work in one department
Department → Employee	One Department has many employees
Patient → Admission	A patient is admitted to the hospital.
Patient → Billing	A patient pays the billing amount
Employee → Receptionist	An employee can be a receptionist.
Employee → Nurse	An employee can be a nurse.

3.3 ER Model

The ER model represents the major entities and their relationships in the Hospital Management System. The diagram identifies Patient–Appointment, Appointment–Doctor, Doctor–Department, Department–Employee, Patient–Admission, Patient–Billing, and Employee–Receptionist/Nurse relationships. The diagram also shows attributes such as PatientID, Name, DOB, Gender, AppointmentID, Date, Time, DoctorID, Specialization, DepartmentID, AdmissionID, BedNo, BillID, Amount, EmployeeID, Shift_Timings, Desk_Number, and Experience_Years.

4.ENTITY RELATIONSHIP DIAGRAM

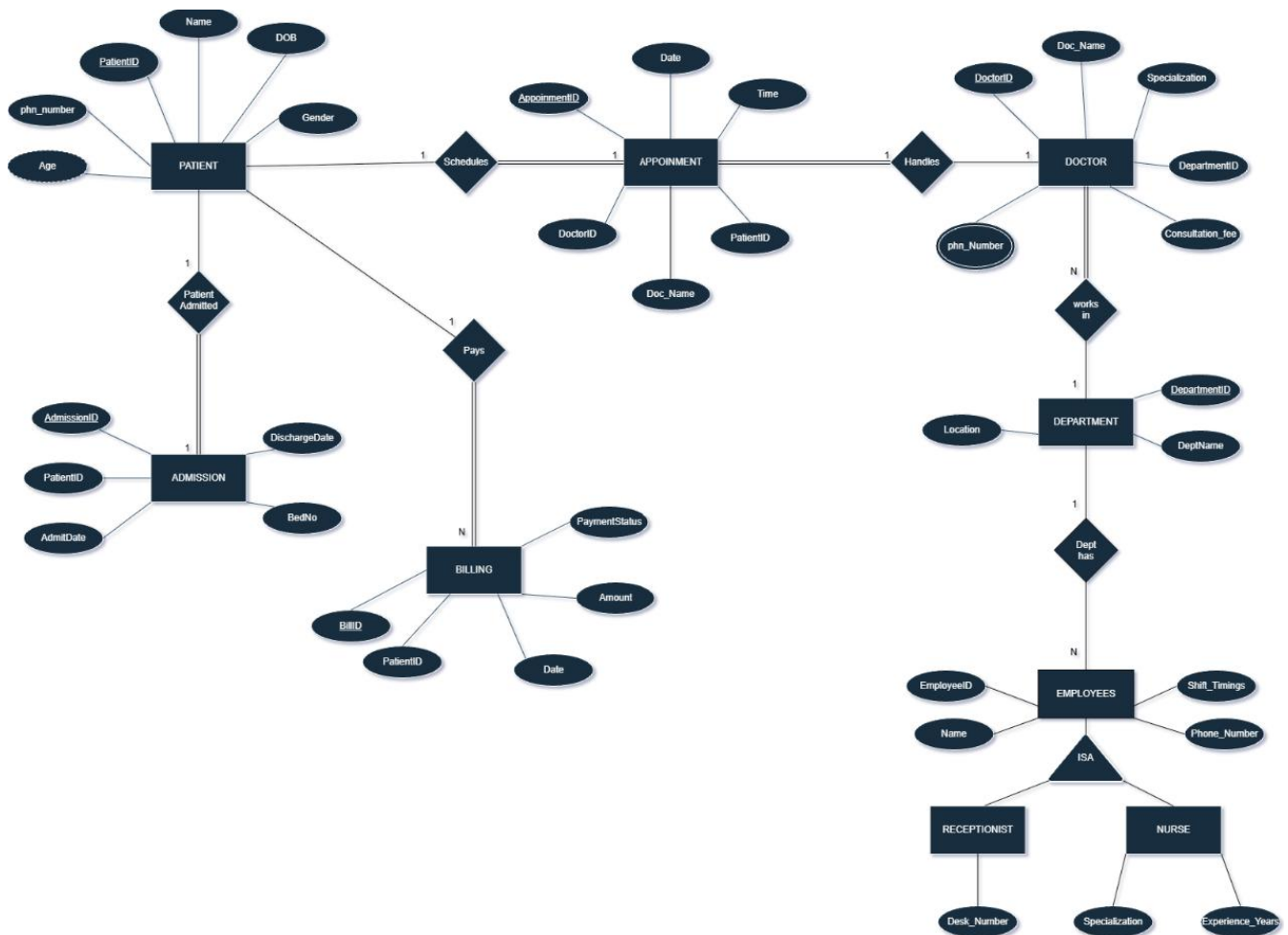


Figure 1: ER Diagram of Hospital Management System Database

The ER diagram summarizes the entity structure and relationships of the Hospital Management System. It represents entities such as Patient, Appointment, Doctor, Department, Admission, Billing, and Employee, with Receptionist and Nurse as specialized employee entities. The diagram shows relationships such as Schedules, Handles, Works In, Department Has, Patient Admitted, and Pays, along with their cardinalities. It also includes attributes such as PatientID, AppointmentID, DoctorID, DepartmentID, AdmissionID, BillID, EmployeeID, Name, Date, Time, Specialization, BedNo, Amount, and PaymentStatus.

5. RELATIONAL SCHEMA

Relation	Relational Schema
PATIENT	PatientID (PK), Name, DOB, Gender, phn_number (UNIQUE)
DEPARTMENT	DepartmentID (PK), DeptName (UNIQUE), Location
DOCTOR	DoctorID (PK), Doc_Name, Specialization, DepartmentID (FK), Consultation_fee
DOCTOR_PHONE	DoctorID (PK, FK), phn_number (PK)
EMPLOYEES	EmployeeID (PK), Name, DepartmentID (FK), Shift_Timings, Phone_Number (UNIQUE)
RECEPTIONIST	EmployeeID (PK, FK), Desk_Number (UNIQUE)
NURSE	EmployeeID (PK, FK), Specialization, Experience_Years
APPOINTMENT	AppointmentID (PK), PatientID (FK, UNIQUE), DoctorID (FK, UNIQUE), Appointment_Date, Appointment_Time
ADMISSION	AdmissionID (PK), PatientID (FK, UNIQUE), AdmitDate, DischargeDate, BedNo
BILLING	BillID (PK), PatientID (FK), Billing_Date, Amount, PaymentStatus

The relational schema is derived directly from the table definitions in the supplied Oracle SQL script. The foreign-key relationships connect doctors and employees to departments, subtype records to EMPLOYEES, appointments to patients and doctors, admissions to patients, and billing records to patients.

6. TABLE DESIGN

6.1 PATIENT Table

Attribute	Data Type	Constraint	Description
PatientID	VARCHAR2(10)	PK	Unique patient identifier
Name	VARCHAR2(60)	NOT NULL	Patient name
DOB	DATE	NOT NULL	Date of birth
Gender	CHAR(1)	NOT NULL, CHECK	M, F, or O
phn_number	VARCHAR2(10)	NOT NULL, UNIQUE, CHECK	10-digit patient phone number

6.2 DEPARTMENT Table

Attribute	Data Type	Constraint	Description
DepartmentID	VARCHAR2(10)	PK	Unique department identifier
DeptName	VARCHAR2(60)	NOT NULL, UNIQUE	Department name
Location	VARCHAR2(100)	NOT NULL	Department location

6.3 DOCTOR Table

Attribute	Data Type	Constraint	Description
DoctorID	VARCHAR2(10)	PK	Unique doctor identifier
Doc Name	VARCHAR2(60)	NOT NULL	Doctor name
Specialization	VARCHAR2(60)	NOT NULL	Medical specialization
DepartmentID	VARCHAR2(10)	NOT NULL, FK	References DEPARTMENT
Consultation_fee	NUMBER(10,2)	NOT NULL, CHECK	Consultation fee greater than zero

6.4 DOCTOR_PHONE Table

Attribute	Data Type	Constraint	Description
DoctorID	VARCHAR2(10)	PK, FK	Doctor reference
phn_number	VARCHAR2(10)	PK, CHECK	Doctor phone number; composite key with DoctorID

6.5 EMPLOYEES Table

Attribute	Data Type	Constraint	Description
EmployeeID	VARCHAR2(10)	PK	Unique employee identifier
Name	VARCHAR2(60)	NOT NULL	Employee real name
DepartmentID	VARCHAR2(10)	NOT NULL, FK	References DEPARTMENT
Shift_Timings	VARCHAR2(20)	NOT NULL, CHECK	Morning, Evening, or Night
Phone_Number	VARCHAR2(10)	NOT NULL, UNIQUE, CHECK	Employee phone number

6.6 RECEPTIONIST Table

Attribute	Data Type	Constraint	Description
EmployeeID	VARCHAR2(10)	PK, FK	References EMPLOYEES
Desk_Number	VARCHAR2(20)	NOT NULL, UNIQUE	Reception desk number

6.7 NURSE Table

Attribute	Data Type	Constraint	Description
EmployeeID	VARCHAR2(10)	PK, FK	References EMPLOYEES
Specialization	VARCHAR2(60)	NOT NULL	Nursing specialization
Experience_Years	NUMBER(2)	NOT NULL, CHECK	Experience from 0 to 50 years

6.8 APPOINTMENT Table

Attribute	Data Type	Constraint	Description
AppointmentID	VARCHAR2(10)	PK	Unique appointment identifier
PatientID	VARCHAR2(10)	NOT NULL, FK, UNIQUE	References PATIENT
DoctorID	VARCHAR2(10)	NOT NULL, FK, UNIQUE	References DOCTOR
Appointment Date	DATE	NOT NULL	Appointment date
Appointment Time	VARCHAR2(5)	NOT NULL	Appointment time

6.9 ADMISSION Table

Attribute	Data Type	Constraint	Description
AdmissionID	VARCHAR2(10)	PK	Unique admission identifier
PatientID	VARCHAR2(10)	NOT NULL, FK, UNIQUE	References PATIENT
AdmitDate	DATE	NOT NULL	Admission date
DischargeDate	DATE	CHECK	Null or not earlier than AdmitDate
BedNo	VARCHAR2(20)	NOT NULL	Assigned bed number

6.10 BILLING Table

Attribute	Data Type	Constraint	Description
BillID	VARCHAR2(10)	PK	Unique bill identifier
PatientID	VARCHAR2(10)	NOT NULL, FK	References PATIENT
Billing Date	DATE	NOT NULL	Billing date
Amount	NUMBER(10,2)	NOT NULL, CHECK	Billing amount greater than zero
PaymentStatus	VARCHAR2(15)	NOT NULL, CHECK	PAID or PENDING

6.11 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	PATIENT.PatientID; DEPARTMENT.DepartmentID; DOCTOR.DoctorID; EMPLOYEES.EmployeeID; subtype IDs; APPOINTMENT.AppointmentID; ADMISSION.AdmissionID; BILLING.BillID.
Composite Primary Key	DOCTOR_PHONE(DoctorID, phn_number).
Foreign Key	Doctor/employee department references; subtype-to-employee references; appointment-to-patient/doctor; admission-to-patient; billing-to-patient.
UNIQUE	Patient phone, department name, employee phone, receptionist desk, appointment patient/doctor, admission patient.
NOT NULL	Required identifiers, names, dates, contact numbers, and operational fields as defined in the SQL.
CHECK	Gender, phone number format, consultation fee, employee shift, nurse experience, admission dates, billing amount, and payment status.

7. NORMALIZATION

The supplied SQL script identifies the Hospital Management System as normalized up to 3NF. Normalization separates independent facts into related relations so that the same information does not need to be repeatedly stored.

7.1 First Normal Form (1NF)

In 1NF, attributes contain atomic values and repeating or multivalued groups are separated. The supplied design separates the multivalued doctor phone attribute into DOCTOR_PHONE, allowing each doctor phone number to be stored as an individual row.

Before 1NF:

PatientID	PatientName	PatientPhone	DoctorID	DoctorName	DoctorPhones	DeptID	DeptName	DeptLocation	AppointmentDate
P001	Ravi Kumar	9876543210	D001	Dr. Anil	9871111111, 9872222222	DP01	Cardiology	Block A	20-09-2026
P002	Sita Rao	9876543211	D002	Dr. Meena	9873333333	DP02	Neurology	Block B	21-09-2026

After 1NF:

DoctorID	phn_number
D001	9871111111
D001	9872222222
D002	9873333333

7.2 Second Normal Form (2NF)

The design separates patient-specific facts from appointment-specific facts. Patient details such as PatientName and PatientPhone are stored in the PATIENT table, while appointment-related attributes such as AppointmentID, PatientID, DoctorID, DoctorName, AppointmentDate, and AppointmentTime are stored in the APPOINTMENT table. PatientID is the primary key of PATIENT, and AppointmentID is the primary key of APPOINTMENT. This separation removes the partial dependency and ensures that both tables satisfy 2NF.

Before 2NF:

PatientID	PatientName	PatientPhone	AppointmentID	DoctorID	DoctorName	AppointmentDate	AppointmentTime
P001	Ravi Kumar	9876543210	A001	D001	Dr. Anil	20-09-2026	10:00
P001	Ravi Kumar	9876543210	A002	D002	Dr. Meena	22-09-2026	11:00
P002	Sita Rao	9876543211	A003	D001	Dr. Anil	23-09-2026	09:30

After 2NF:

TABLE 1 — PATIENT

PatientID (PK)	PatientName	PatientPhone
P001	Ravi Kumar	9876543210
P002	Sita Rao	9876543211

TABLE 2 — APPOINTMENT

AppointmentID (PK)	PatientID (FK)	DoctorID	DoctorName	AppointmentDate	AppointmentTime
A001	P001	D001	Dr. Anil	20-09-2026	10:00
A002	P001	D002	Dr. Meena	22-09-2026	11:00
A003	P002	D001	Dr. Anil	23-09-2026	09:30

7.3 Third Normal Form (3NF)

In 3NF, non-key attributes should not depend transitively on another non-key attribute. Doctor details are stored in the separate DOCTOR table instead of being repeated in APPOINTMENT. DoctorID is retained in APPOINTMENT as a foreign key, while DoctorName is stored only in DOCTOR. Thus, the transitive dependency AppointmentID → DoctorID → DoctorName is removed, resulting in the 3NF tables: PATIENT, DOCTOR, and APPOINTMENT.

Before 3NF :

<u>AppointmentID (PK)</u>	PatientID (FK)	DoctorID	DoctorName	AppointmentDate	AppointmentTime
A001	P001	D001	Dr. Anil	20-09-2026	10:00
A002	P001	D002	Dr. Meena	22-09-2026	11:00
A003	P002	D001	Dr. Anil	23-09-2026	09:30

After 3NF:

TABLE 1 — PATIENT

<u>PatientID (PK)</u>	<u>PatientName</u>	<u>PatientPhone</u>
P001	Ravi Kumar	9876543210
P002	Sita Rao	9876543211

TABLE 2 — Doctor

<u>DoctorID (PK)</u>	<u>DoctorName</u>	
D001	Dr. Anil	
D002	Dr. Meena	

TABLE 3 — Appointment

<u>AppointmentID (PK)</u>	<u>PatientID (FK)</u>	<u>DoctorID (FK)</u>	<u>AppointmentDate</u>	<u>AppointmentTime</u>
A001	P001	D001	20-09-2026	10:00
A002	P001	D002	22-09-2026	11:00
A003	P002	D001	23-09-2026	09:30

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following representative DDL statements are taken from the supplied Oracle SQL implementation. The complete SQL script contains the definitions for all ten tables.

```
CREATE TABLE PATIENT (
    PatientID    VARCHAR2(10),
    Name         VARCHAR2(60) NOT NULL,
    DOB          DATE NOT NULL,
    Gender       CHAR(1) NOT NULL,
    phn_number   VARCHAR2(10) NOT NULL,
    CONSTRAINT pk_patient PRIMARY KEY (PatientID),
    CONSTRAINT uq_patient_phone UNIQUE (phn_number),
    CONSTRAINT chk_patient_gender CHECK (Gender IN ('M','F','O'))
);
CREATE TABLE DEPARTMENT (
    DepartmentID VARCHAR2(10),
    DeptName     VARCHAR2(60) NOT NULL,
    Location     VARCHAR2(100) NOT NULL,
    CONSTRAINT pk_department PRIMARY KEY (DepartmentID),
    CONSTRAINT uq_department_name UNIQUE (DeptName)
);
CREATE TABLE DOCTOR (
    DoctorID     VARCHAR2(10),
    Doc_Name     VARCHAR2(60) NOT NULL,
    Specialization VARCHAR2(60) NOT NULL,
    DepartmentID VARCHAR2(10) NOT NULL,
    Consultation_fee NUMBER(10,2) NOT NULL,
    CONSTRAINT pk_doctor PRIMARY KEY (DoctorID),
    CONSTRAINT fk_doctor_department
        FOREIGN KEY (DepartmentID) REFERENCES DEPARTMENT(DepartmentID),
    CONSTRAINT chk_doctor_fee CHECK (Consultation_fee > 0)
);
CREATE TABLE DOCTOR_PHONE (
    DoctorID     VARCHAR2(10),
    phn_number   VARCHAR2(10),
    CONSTRAINT pk_doctor_phone PRIMARY KEY (DoctorID, phn_number),
    CONSTRAINT fk_doctor_phone
        FOREIGN KEY (DoctorID) REFERENCES DOCTOR(DoctorID)
);
CREATE TABLE EMPLOYEES (
    EmployeeID   VARCHAR2(10),
    Name         VARCHAR2(60) NOT NULL,
    DepartmentID VARCHAR2(10) NOT NULL,
    Shift_Timings VARCHAR2(20) NOT NULL,
    Phone_Number VARCHAR2(10) NOT NULL,
    CONSTRAINT pk_employees PRIMARY KEY (EmployeeID),
    CONSTRAINT fk_employee_department
        FOREIGN KEY (DepartmentID) REFERENCES DEPARTMENT(DepartmentID)
);
CREATE TABLE RECEPTIONIST (
    EmployeeID   VARCHAR2(10),
```

```

    Desk_Number VARCHAR2(20) NOT NULL,
    CONSTRAINT pk_receptionist PRIMARY KEY (EmployeeID),
    CONSTRAINT fk_receptionist_employee
        FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEES(EmployeeID)
);
CREATE TABLE NURSE (
    EmployeeID      VARCHAR2(10),
    Specialization  VARCHAR2(60) NOT NULL,
    Experience_Years NUMBER(2) NOT NULL,
    CONSTRAINT pk_nurse PRIMARY KEY (EmployeeID),
    CONSTRAINT fk_nurse_employee
        FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEES(EmployeeID)
);
CREATE TABLE APPOINTMENT (
    AppointmentID    VARCHAR2(10),
    PatientID        VARCHAR2(10) NOT NULL,
    DoctorID         VARCHAR2(10) NOT NULL,
    Appointment_Date DATE NOT NULL,
    Appointment_Time VARCHAR2(5) NOT NULL,
    CONSTRAINT pk_appointment PRIMARY KEY (AppointmentID),
    CONSTRAINT fk_appointment_patient
        FOREIGN KEY (PatientID) REFERENCES PATIENT(PatientID),
    CONSTRAINT fk_appointment_doctor
        FOREIGN KEY (DoctorID) REFERENCES DOCTOR(DoctorID),
    CONSTRAINT uq_appointment_patient UNIQUE (PatientID),
    CONSTRAINT uq_appointment_doctor UNIQUE (DoctorID)
);
CREATE TABLE ADMISSION (
    AdmissionID      VARCHAR2(10),
    PatientID        VARCHAR2(10) NOT NULL,
    AdmitDate        DATE NOT NULL,
    DischargeDate    DATE,
    BedNo            VARCHAR2(20) NOT NULL,
    CONSTRAINT pk_admission PRIMARY KEY (AdmissionID),
    CONSTRAINT fk_admission_patient
        FOREIGN KEY (PatientID) REFERENCES PATIENT(PatientID),
    CONSTRAINT uq_admission_patient UNIQUE (PatientID)
);
CREATE TABLE BILLING (
    BillID           VARCHAR2(10),
    PatientID        VARCHAR2(10) NOT NULL,
    Billing_Date      DATE NOT NULL,
    Amount           NUMBER(10,2) NOT NULL,
    PaymentStatus    VARCHAR2(15) NOT NULL,
    CONSTRAINT pk_billing PRIMARY KEY (BillID),
    CONSTRAINT fk_billing_patient
        FOREIGN KEY (PatientID) REFERENCES PATIENT(PatientID)
);

```

8.2 DML – Data Manipulation Language

The supplied script inserts sample data using Oracle INSERT ALL statements. The implemented dataset includes realistic Indian patient names, 15 departments, 15 doctors, 15 doctor phone records, 30 employees, 15 receptionists, 15 nurses, 15 appointments, 15 admissions, and 15 billing records.

```
INSERT INTO PATIENT
VALUES ('P001','Aarav Reddy',
       TO_DATE('12-01-2005','DD-MM-YYYY'),'M','9876500001');

INSERT INTO DEPARTMENT
VALUES ('D001','Cardiology','Block A - First Floor');

INSERT INTO DOCTOR
VALUES ('DR001','Dr. Suresh Rao','Cardiologist','D001',800);
```

8.3 DQL – Data Query Language

The project includes SELECT queries for retrieval, joins, filtering, aggregation, and reporting. Representative examples are shown below.

```
SELECT * FROM PATIENT ORDER BY PatientID;

SELECT d.DoctorID, d.Doc_Name, d.Specialization, dep.DeptName
FROM DOCTOR d
JOIN DEPARTMENT dep
ON d.DepartmentID = dep.DepartmentID
ORDER BY d.DoctorID;

SELECT SUM(Amount) AS Total_Billing
FROM BILLING;
```

9. SQL QUERIES & SCREENSHOTS

1. Display All Patients

```
SELECT * FROM PATIENT;
```

OutPut:

PATIENTID	NAME	DOB	G	PHN_NUMBER	AGE
P001	Aarav Reddy	12-JAN-05	M	9876500001	21
P002	Ananya Sharma	24-FEB-04	F	9876500002	22
P003	Rahul Varma	05-MAR-03	M	9876500003	23
P004	Sneha Rao	18-APR-02	F	9876500004	24
P005	Vikram Singh	30-MAY-01	M	9876500005	25
P006	Kavya Nair	11-JUN-05	F	9876500006	21
P007	Aditya Iyer	22-JUL-00	M	9876500007	26
P008	Priya Menon	09-AUG-04	F	9876500008	22
P009	Rohit Gupta	15-SEP-03	M	9876500009	23
P010	Divya Patel	27-OCT-02	F	9876500010	23
P011	Arjun Kumar	03-NOV-01	M	9876500011	24
PATIENTID	NAME	DOB	G	PHN_NUMBER	AGE
P012	Meera Das	19-DEC-05	F	9876500012	20
P013	Karthik Reddy	07-JAN-00	M	9876500013	26
P014	Pooja Shah	16-FEB-04	F	9876500014	22
P015	Sandeep Joshi	28-MAR-03	M	9876500015	23

2. Display Patient Names and Phone Numbers

```
SELECT Name, phn_number FROM PATIENT;
```

Output:

NAME	PHN_NUMBER
Aarav Reddy	9876500001
Ananya Sharma	9876500002
Rahul Varma	9876500003
Sneha Rao	9876500004
Vikram Singh	9876500005
Kavya Nair	9876500006
Aditya Iyer	9876500007
Priya Menon	9876500008
Rohit Gupta	9876500009
Divya Patel	9876500010
Arjun Kumar	9876500011
NAME	PHN_NUMBER
Meera Das	9876500012
Karthik Reddy	9876500013
Pooja Shah	9876500014
Sandeep Joshi	9876500015

3. Find Female Patients

SELECT * FROM PATIENT WHERE Gender = 'F';

Output:

PATIENTID	NAME	DOB	G	PHN_NUMBER	AGE
P002	Ananya Sharma	24-FEB-04	F	9876500002	22
P004	Sneha Rao	18-APR-02	F	9876500004	24
P006	Kavya Nair	11-JUN-05	F	9876500006	21
P008	Priya Menon	09-AUG-04	F	9876500008	22
P010	Divya Patel	27-OCT-02	F	9876500010	23
P012	Meera Das	19-DEC-05	F	9876500012	20
P014	Pooja Shah	16-FEB-04	F	9876500014	22

4. Find Patients Born After 2000

SELECT *FROM PATIENT WHERE DOB > DATE '2000-01-01';

Output:

PATIENTID	NAME	DOB	G	PHN_NUMBER	AGE
P001	Aarav Reddy	12-JAN-05	M	9876500001	21
P002	Ananya Sharma	24-FEB-04	F	9876500002	22
P003	Rahul Varma	05-MAR-03	M	9876500003	23
P004	Sneha Rao	18-APR-02	F	9876500004	24
P005	Vikram Singh	30-MAY-01	M	9876500005	25
P006	Kavya Nair	11-JUN-05	F	9876500006	21
P007	Aditya Iyer	22-JUL-00	M	9876500007	26
P008	Priya Menon	09-AUG-04	F	9876500008	22
P009	Rohit Gupta	15-SEP-03	M	9876500009	23
P010	Divya Patel	27-OCT-02	F	9876500010	23
P011	Arjun Kumar	03-NOV-01	M	9876500011	24
PATIENTID	NAME	DOB	G	PHN_NUMBER	AGE
P012	Meera Das	19-DEC-05	F	9876500012	20
P013	Karthik Reddy	07-JAN-00	M	9876500013	26
P014	Pooja Shah	16-FEB-04	F	9876500014	22
P015	Sandeep Joshi	28-MAR-03	M	9876500015	23

5. Sort Patients by Name

```
SELECT * FROM PATIENT ORDER BY Name ASC;
```

Output:

PATIENTID	NAME	DOB	G	PHN_NUMBER	AGE
P001	Aarav Reddy	12-JAN-05	M	9876500001	21
P007	Aditya Iyer	22-JUL-00	M	9876500007	26
P002	Ananya Sharma	24-FEB-04	F	9876500002	22
P011	Arjun Kumar	03-NOV-01	M	9876500011	24
P010	Divya Patel	27-OCT-02	F	9876500010	23
P013	Karthik Reddy	07-JAN-00	M	9876500013	26
P006	Kavya Nair	11-JUN-05	F	9876500006	21
P012	Meera Das	19-DEC-05	F	9876500012	20
P014	Pooja Shah	16-FEB-04	F	9876500014	22
P008	Priya Menon	09-AUG-04	F	9876500008	22
P003	Rahul Varma	05-MAR-03	M	9876500003	23
PATIENTID	NAME	DOB	G	PHN_NUMBER	AGE
P009	Rohit Gupta	15-SEP-03	M	9876500009	23
P015	Sandeep Joshi	28-MAR-03	M	9876500015	23
P004	Sneha Rao	18-APR-02	F	9876500004	24
P005	Vikram Singh	30-MAY-01	M	9876500005	25

6. Count Total Patients

```
SELECT COUNT(*) AS Total_Patients FROM PATIENT;
```

Output:

TOTAL_PATIENTS
15

7. Display Unique Genders

```
SELECT DISTINCT Gender FROM PATIENT;
```

Output:

Advanced SQL Queries

1. Patient Details with Appointment Information

```
SELECT p.PatientID, p.Name, a.AppointmentID, a.Appointment_Date
FROM PATIENT p JOIN APPOINTMENT a
ON p.PatientID = a.PatientID;
```

Output:

PATIENTID	NAME	APPOINTMEN	APPOINTME
P001	Aarav Reddy	A001	01-OCT-26
P002	Ananya Sharma	A002	02-OCT-26
P003	Rahul Varma	A003	03-OCT-26
P004	Sneha Rao	A004	04-OCT-26
P005	Vikram Singh	A005	05-OCT-26
P006	Kavya Nair	A006	06-OCT-26
P007	Aditya Iyer	A007	07-OCT-26
P008	Priya Menon	A008	08-OCT-26
P009	Rohit Gupta	A009	09-OCT-26
P010	Divya Patel	A010	10-OCT-26
P011	Arjun Kumar	A011	11-OCT-26
PATIENTID	NAME	APPOINTMEN	APPOINTME
P012	Meera Das	A012	12-OCT-26
P013	Karthik Reddy	A013	13-OCT-26
P014	Pooja Shah	A014	14-OCT-26
P015	Sandeep Joshi	A015	15-OCT-26

2. Doctors and Their Departments

```
SELECT d.DoctorID, d.Doc_Name AS Doctor_Name, dept.DeptName
FROM DOCTOR d JOIN DEPARTMENT dept
ON d.DepartmentID = dept.DepartmentID;
```

Output:

DOCTORID	DOCTOR_NAME	DEPTNAME
DR001	Dr. Suresh Rao	Cardiology
DR002	Dr. Neha Sharma	Neurology
DR003	Dr. Rajesh Kumar	Orthopedics
DR004	Dr. Anjali Patel	Pediatrics
DR005	Dr. Vivek Singh	Dermatology
DR006	Dr. Meena Nair	ENT
DR007	Dr. Ramesh Iyer	General Medicine
DR008	Dr. Sanjay Verma	General Surgery
DR009	Dr. Lakshmi Rao	Ophthalmology
DR010	Dr. Kavitha Reddy	Gynecology
DR011	Dr. Arvind Gupta	Urology
DR012	Dr. Priyanka Menon	Oncology
DR013	Dr. Mahesh Shah	Psychiatry
DR014	Dr. Nitin Joshi	Radiology
DR015	Dr. Farhan Khan	Emergency Medicine

3. Number of Appointments for Each Doctor

```
SELECT d.DoctorID, d.Doc_Name AS Doctor_Name, COUNT(a.AppointmentID) AS
Total_Appointments
```

```
FROM DOCTOR d LEFT JOIN APPOINTMENT a ON d.DoctorID = a.DoctorID
```

```
GROUP BY d.DoctorID, d.Doc_Name;
```

Output:

DOCTORID	DOCTOR_NAME	TOTAL_APPOINTMENTS
DR005	Dr. Vivek Singh	1
DR009	Dr. Lakshmi Rao	1
DR004	Dr. Anjali Patel	1
DR011	Dr. Arvind Gupta	1
DR015	Dr. Farhan Khan	1
DR002	Dr. Neha Sharma	1
DR003	Dr. Rajesh Kumar	1
DR008	Dr. Sanjay Verma	1
DR010	Dr. Kavitha Reddy	1
DR007	Dr. Ramesh Iyer	1
DR013	Dr. Mahesh Shah	1
DR006	Dr. Meena Nair	1
DR014	Dr. Nitin Joshi	1
DR001	Dr. Suresh Rao	1
DR012	Dr. Priyanka Menon	1

4. Patients and Their Total Bills

```
SELECT p.PatientID, p.Name, SUM(b.Amount) AS Total_Bill FROM PATIENT p
JOIN BILLING b ON p.PatientID = b.PatientID GROUP BY p.PatientID, p.Name;
```

Output:

PATIENTID	NAME	TOTAL_BILL
P001	Aarav Reddy	5350
P004	Sneha Rao	6400
P005	Vikram Singh	6750
P013	Karthik Reddy	9550
P010	Divya Patel	8500
P014	Pooja Shah	9900
P007	Aditya Iyer	7450
P003	Rahul Varma	6050
P006	Kavya Nair	7100
P009	Rohit Gupta	8150
P015	Sandeep Joshi	10250

PATIENTID	NAME	TOTAL_BILL
P002	Ananya Sharma	5700
P008	Priya Menon	7800
P011	Arjun Kumar	8850
P012	Meera Das	9200

5. Doctors with More Appointments Than Average

```
SELECT d.DoctorID, d.Doc_Name AS Doctor_Name, d.Specialization,d.Consultation_fee
FROM DOCTOR d WHERE d.Consultation_fee > (SELECT AVG(Consultation_fee) FROM
DOCTOR)
ORDER BY d.Consultation_fee DESC;
```

Output:

DOCTORID	DOCTOR_NAME	SPECIALIZATION	CONSULTATION_FEE
DR012	Dr. Priyanka Menon	Oncologist	1000
DR008	Dr. Sanjay Verma	General Surgeon	900
DR014	Dr. Nitin Joshi	Radiologist	900
DR002	Dr. Neha Sharma	Neurologist	850
DR011	Dr. Arvind Gupta	Urologist	850
DR001	Dr. Suresh Rao	Cardiologist	800
DR010	Dr. Kavitha Reddy	Gynecologist	800

6. Patients with bills above the average bill amount

```
SELECT p.PatientID, p.Name AS Patient_Name, b.Amount, b.PaymentStatus
FROM PATIENT p JOIN BILLING b ON p.PatientID = b.PatientID
WHERE b.Amount > (SELECT AVG(Amount)FROM BILLING)
ORDER BY b.Amount DESC;
```

PATIENTID	PATIENT_NAME	AMOUNT	PAYMENTSTATUS
P015	Sandeep Joshi	10250	PAID
P014	Pooja Shah	9900	PAID
P013	Karthik Reddy	9550	PAID
P012	Meera Das	9200	PENDING
P011	Arjun Kumar	8850	PAID
P010	Divya Patel	8500	PAID
P009	Rohit Gupta	8150	PAID

7. Departments with more than one employee

```
SELECT d.DepartmentID, d.DeptName, COUNT(e.EmployeeID) AS Employee_Count
FROM DEPARTMENT d JOIN EMPLOYEES e ON d.DepartmentID = e.DepartmentID
GROUP BY d.DepartmentID, d.DeptName HAVING COUNT(e.EmployeeID) > 1
ORDER BY Employee_Count DESC;
```

Output:

DEPARTMENT	DEPTNAME	EMPLOYEE_COUNT
D003	Orthopedics	2
D006	ENT	2
D008	General Surgery	2
D002	Neurology	2

8. Departments with their total employee count and average employee experience

```
SELECT d.DepartmentID, d.DeptName, COUNT(e.EmployeeID) AS Total_Employees,
ROUND(AVG(n.Experience_Years), 2) AS Avg_Experience FROM DEPARTMENT d JOIN EMPLOYEES e
ON d.DepartmentID = e.DepartmentID JOIN NURSE n ON e.EmployeeID = n.EmployeeID
```

GROUP BY d.DepartmentID, d.DeptName HAVING COUNT(e.EmployeeID) >= 1

ORDER BY Avg_Experience DESC;

Output:

DEPARTMENT	DEPTNAME	TOTAL_EMPLOYEES	AVG_EXPERIENCE
D011	Urology	1	8
D006	ENT	1	7
D004	Pediatrics	1	6
D010	Gynecology	1	6
D003	Orthopedics	1	5
D009	Ophthalmology	1	5
D015	Emergency Medicine	1	5
D002	Neurology	1	4
D008	General Surgery	1	4
D014	Radiology	1	4
D001	Cardiology	1	3
D007	General Medicine	1	3
D013	Psychiatry	1	3
D005	Dermatology	1	2
D012	Oncology	1	2

10.Conclusion

The Hospital Management Database Application successfully demonstrates the practical implementation of Database Management System concepts in a healthcare environment. The system provides a centralized and structured database for managing patients, doctors, departments, appointments, admissions, billing, nurses, and receptionists.

The database is designed using an Entity-Relationship model and converted into relational tables with appropriate primary keys, foreign keys, unique constraints, NOT NULL constraints, and CHECK constraints. The database is normalized up to Third Normal Form (3NF), which helps reduce data redundancy and maintain data consistency.

SQL operations such as DDL, DML, and DQL are used to create tables, insert data, and retrieve useful information through basic and advanced queries. The use of joins, aggregation, filtering, grouping, and subqueries demonstrates how hospital information can be efficiently accessed and analysed.

Overall, the project provides a strong database foundation for a Hospital Management System and demonstrates how DBMS concepts can be applied to organize hospital information, maintain relationships between different entities, and support efficient data retrieval and reporting.

11. FUTURE ENHANCEMENTS

- Role-based access for administrators, doctors, nurses, receptionists, and other authorized hospital staff.
- Patient appointment, admission, discharge, and bed management.
- Pharmacy and medicine inventory management.
- Laboratory test management and electronic medical record integration.
- Automated billing, payment tracking, and insurance management.
- Automated reports and analytics dashboards for patients, appointments, admissions, and billing.
- Backup, recovery, audit logging, and stronger database security.

12. References

1. Database Systems, Ramez Elmasri, Shamkant B. Navathe, 7th Edition, Pearson, ISBN: 978-0137504277.
2. Database Systems, Carlos Coronel, Steven Morris, Peter Rob, 10th Edition, Cengage, ISBN: 978-0357517864.
3. Introduction to Database Systems, C J Date, Pearson, 9th Edition, ISBN: 978-0133970777.

